

Deep Networks

Activation Functions, Error Functions, Backpropagation and Automatic
Differentiation

Introduction to Deep Learning · Class 11

Outline

From What to How

Activation Functions

Error Functions, Revisited

Backpropagation

Automatic Differentiation

Putting It Together

From What to How

Notation used throughout this course

$\mathbf{x}, y, \hat{y}$	input, target, prediction	$\delta_j^{(l)}$	$\partial E_n / \partial a_j^{(l)}$
$a_j^{(l)}$	pre-activation, layer l	L, M, W	depth, width, parameter count
$z_j^{(l)} = h(a_j^{(l)})$	activation	$\mathbf{J}$	Jacobian $\partial \hat{\mathbf{y}} / \partial \mathbf{x}$
$w_{ji}^{(l)}$	weight into unit j from unit i	η	learning rate
$h(\cdot), h'(\cdot)$	activation and its derivative	$\dot{v}, \bar{v}$	tangent, adjoint (§5)
$E(\mathbf{w}), E_n$	total and per-example error	$\odot$	element-wise product

The one new symbol is $\delta_j^{(l)}$, the **error** at a unit. Everything in §4 is a rule for computing it.

Six lectures of *what*; now *how*

- Classes 5–6: a network is regression on **learned features**, and the error function follows from a **distribution** over the output
- Classes 7–10: depth composes, universality holds, and **depth is exponentially more efficient** than width
- Every one of those results is about the **existence** of good weights

Training needs $\nabla E(\mathbf{w})$ — one number for each of W parameters, at every step.
Getting it is today.

Four questions

	Question	Where
1	Which h should a hidden unit use, and why does it matter for <i>training</i> ?	§2
2	Which error functions exist beyond the three from Class 6?	§3
3	How do we get all W derivatives at once?	§4
4	Is backpropagation a special trick, or an instance of something general?	§5

The short answers: choose h for its **derivative**; choose E for the **distribution**; get the gradient by the **chain rule on a graph**; and that rule is **reverse-mode automatic differentiation**.

Activation Functions

What the activation is for

We have used ReLU without ever asking what the alternatives cost. Training is what settles the question.

$$a_j^{(l)} = \sum_i w_{ji}^{(l)} z_i^{(l-1)} + b_j^{(l)}, \quad z_j^{(l)} = h(a_j^{(l)})$$

- Without h the whole network collapses to one affine map (Class 5)
- Any non-polynomial h gives universality (Class 9) — so **expressivity does not choose h for us**
- What does choose it: the **derivative h'** , because that is the factor every layer contributes to the gradient

FORWARD h decides what the layer *computes*

BACKWARD h' decides what the layer *transmits*

The activation zoo: saturating and hard

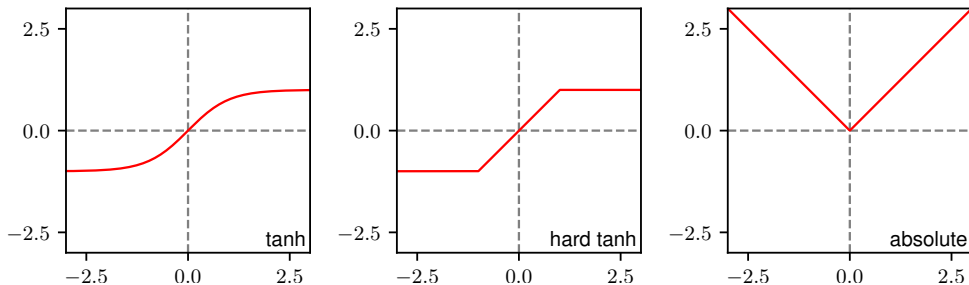


Figure from Bishop & Bishop (2024), Fig. 6.12: the first two are bounded — hence saturation.

The activation zoo: the rectifier family

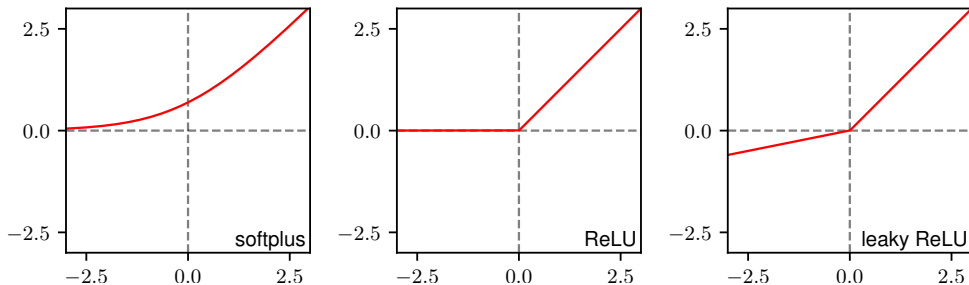
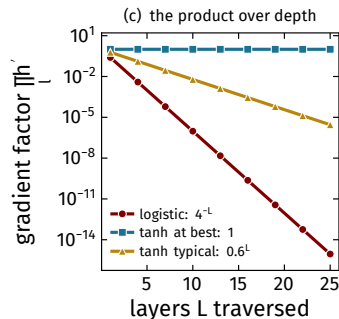
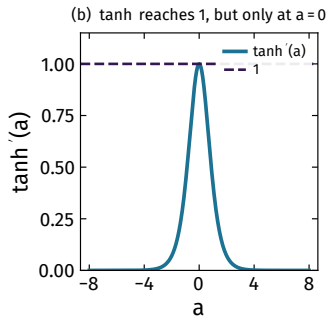
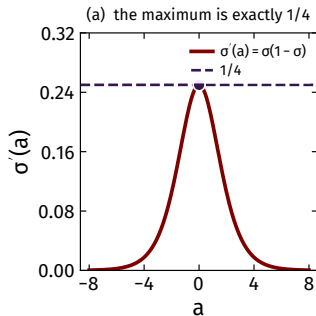


Figure from Bishop & Bishop (2024), Fig. 6.12: all three are unbounded above.

What backpropagation actually uses



(a) the logistic derivative peaks at exactly $1/4$. (b) $\tanh'$ reaches 1, but only at the origin. (c) the product over depth — 4^{-L} is hopeless, and even a typical $\tanh$ factor of 0.6 costs four orders of magnitude by $L = 20$.

Why the logistic sigmoid fails at depth

Proposition 1 (The sigmoid derivative is bounded by $1/4$)

For all a , $\sigma'(a) = \sigma(a)(1 - \sigma(a)) \leq \frac{1}{4}$, with equality only at $a = 0$.

- Write $u = \sigma(a) \in (0, 1)$, so $\sigma' = u(1 - u)$
- $u(1 - u) = \frac{1}{4} - (u - \frac{1}{2})^2$, a downward parabola in u
- Hence $u(1 - u) \leq \frac{1}{4}$, with equality iff $u = \frac{1}{2}$, i.e. $a = 0$

□

Corollary 2 (Sigmoid networks vanish)

Crossing L sigmoid layers multiplies the backpropagated signal by at most 4^{-L} . At $L = 10$ that is under 10^{-6} before any weight is taken into account.

Why the logistic sigmoid fails at depth

Proposition 1 (The sigmoid derivative is bounded by $1/4$)

For all a , $\sigma'(a) = \sigma(a)(1 - \sigma(a)) \leq \frac{1}{4}$, with equality only at $a = 0$.

- Write $u = \sigma(a) \in (0, 1)$, so $\sigma' = u(1 - u)$
- $u(1 - u) = \frac{1}{4} - (u - \frac{1}{2})^2$, **a downward parabola in u**
- Hence $u(1 - u) \leq \frac{1}{4}$, with equality iff $u = \frac{1}{2}$, i.e. $a = 0$

□

Corollary 2 (Sigmoid networks vanish)

Crossing L sigmoid layers multiplies the backpropagated signal by at most 4^{-L} . At $L = 10$ that is under 10^{-6} before any weight is taken into account.

Why the logistic sigmoid fails at depth

Proposition 1 (The sigmoid derivative is bounded by $1/4$)

For all a , $\sigma'(a) = \sigma(a)(1 - \sigma(a)) \leq \frac{1}{4}$, with equality only at $a = 0$.

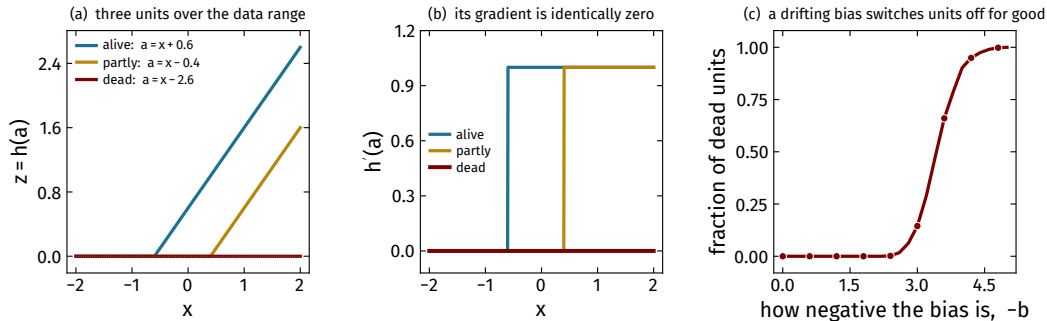
- Write $u = \sigma(a) \in (0, 1)$, so $\sigma' = u(1 - u)$
- $u(1 - u) = \frac{1}{4} - (u - \frac{1}{2})^2$, a downward parabola in u
- Hence $u(1 - u) \leq \frac{1}{4}$, with equality iff $u = \frac{1}{2}$, i.e. $a = 0$

□

Corollary 2 (Sigmoid networks vanish)

Crossing L sigmoid layers multiplies the backpropagated signal by at most 4^{-L} . At $L = 10$ that is under 10^{-6} before any weight is taken into account.

The rectifier and its derivative



(a) three ReLU units over the data range: one active throughout, one on part of it, one never active. (b) their derivatives — the dead unit's is identically zero. (c) as a bias drifts negative, the fraction of permanently dead units climbs.

The rectifier's own failure mode

Proposition 3 (Dead units are permanent)

*If a ReLU unit has $a_j(\mathbf{x}_n) \leq 0$ for **every** training input, then $z_j \equiv 0$ and $\partial E / \partial w_{ji} = 0$ for all i . No gradient step can change its weights, so it stays dead for the rest of training.*

Proof. $\partial E_n / \partial w_{ji} = \delta_j z_i$ and $\delta_j = h'(a_j) \sum_k w_{kj} \delta_k$. For $a_j < 0$, $h'(a_j) = 0$, so $\delta_j = 0$ for every n , hence every derivative into the unit vanishes. □

- ReLU trades **saturation on one side** for **exact zero on the other**
- Leaky ReLU, ELU, GELU and SiLU all keep a small non-zero slope for $a < 0$ for exactly this reason

Choosing an activation

Activation	h' range	Strength	Weakness
logistic	$(0, \frac{1}{4}]$	bounded output	vanishes at depth
tanh	$(0, 1]$	zero-centred	still saturates
ReLU	$\{0, 1\}$	no saturation for $a > 0$, cheap	dead units, not differentiable at 0
leaky ReLU	$\{s, 1\}$	never exactly zero	one more hyperparameter
ELU	$(0, 1]$	smooth, saturates below	exponential to evaluate
GELU / SiLU	$(-0.1, 1.1)$	smooth, the current default	no simple intuition

Read the second column first: an activation is chosen for the **gradient it lets through**, not the shape it draws.

Error Functions, Revisited

What Class 6 established

The output end of the network was settled in Class 6. One slide to recall it, then the cases it did not cover.

Task	Distribution	Output activation	Error
Regression	Gaussian	identity	sum of squares
Binary	Bernoulli	sigmoid	binary cross-entropy
Multi-class	categorical	softmax	cross-entropy

Choose the distribution; the output activation and the error function are then **determined**. And in all three cases $\partial E_n / \partial a_k = \hat{y}_k - y_k$.

That last fact looked like a coincidence three times over. It is not.

Why the output gradient is always $\hat{y} - y$

Theorem 4 (Canonical link)

Let the output distribution be an exponential family $p(y \mid \theta) = \exp\{\theta y - A(\theta) + c(y)\}$, let the network's output pre-activation be its natural parameter, $\theta = a$, and let $E_n = -\ln p(y_n \mid a_n)$. Then

$$\frac{\partial E_n}{\partial a_n} = \hat{y}_n - y_n, \quad \hat{y}_n = A'(a_n) = \mathbb{E}[y \mid a_n].$$

- $E_n = -\theta y_n + A(\theta) - c(y_n)$ with $\theta = a_n$
- Differentiate: $\partial E_n / \partial a_n = -y_n + A'(a_n)$
- For an exponential family $A'(\theta)$ is the mean — exactly what the output activation produces $\square$
- Gaussian gives the identity, Bernoulli the sigmoid, categorical the softmax — the three cases of Class 6 are one theorem

Why the output gradient is always $\hat{y} - y$

Theorem 4 (Canonical link)

Let the output distribution be an exponential family $p(y \mid \theta) = \exp\{\theta y - A(\theta) + c(y)\}$, let the network's output pre-activation be its natural parameter, $\theta = a$, and let $E_n = -\ln p(y_n \mid a_n)$. Then

$$\frac{\partial E_n}{\partial a_n} = \hat{y}_n - y_n, \quad \hat{y}_n = A'(a_n) = \mathbb{E}[y \mid a_n].$$

- $E_n = -\theta y_n + A(\theta) - c(y_n)$ with $\theta = a_n$
- **Differentiate:** $\partial E_n / \partial a_n = -y_n + A'(a_n)$
- For an exponential family $A'(\theta)$ is the mean — exactly what the output activation produces $\square$
- Gaussian gives the identity, Bernoulli the sigmoid, categorical the softmax — **the three cases of Class 6 are one theorem**

Why the output gradient is always $\hat{y} - y$

Theorem 4 (Canonical link)

Let the output distribution be an exponential family $p(y \mid \theta) = \exp\{\theta y - A(\theta) + c(y)\}$, let the network's output pre-activation be its natural parameter, $\theta = a$, and let $E_n = -\ln p(y_n \mid a_n)$. Then

$$\frac{\partial E_n}{\partial a_n} = \hat{y}_n - y_n, \quad \hat{y}_n = A'(a_n) = \mathbb{E}[y \mid a_n].$$

- $E_n = -\theta y_n + A(\theta) - c(y_n)$ with $\theta = a_n$
- Differentiate: $\partial E_n / \partial a_n = -y_n + A'(a_n)$
- **For an exponential family $A'(\theta)$ is the mean — exactly what the output activation produces** $\square$
- Gaussian gives the identity, Bernoulli the sigmoid, categorical the softmax — the three cases of Class 6 are one theorem

Why the output gradient is always $\hat{y} - y$

Theorem 4 (Canonical link)

Let the output distribution be an exponential family $p(y \mid \theta) = \exp\{\theta y - A(\theta) + c(y)\}$, let the network's output pre-activation be its natural parameter, $\theta = a$, and let $E_n = -\ln p(y_n \mid a_n)$. Then

$$\frac{\partial E_n}{\partial a_n} = \hat{y}_n - y_n, \quad \hat{y}_n = A'(a_n) = \mathbb{E}[y \mid a_n].$$

- $E_n = -\theta y_n + A(\theta) - c(y_n)$ with $\theta = a_n$
- Differentiate: $\partial E_n / \partial a_n = -y_n + A'(a_n)$
- For an exponential family $A'(\theta)$ is the mean — exactly what the output activation produces $\square$
- **Gaussian gives the identity, Bernoulli the sigmoid, categorical the softmax — the three cases of Class 6 are one theorem**

Why this matters for training

- The output layer contributes **no h' factor at all**: the A'' from the link and the $1/\text{var}$ from the error cancel
- Mismatch the pair — squared error on a sigmoid output — and the factor comes back as $\sigma'(a)$, which we now know is at most $1/4$ (Class 6, and §2 today)
- So the canonical pairing is not aesthetics: it removes one source of vanishing gradient at the point where the signal enters the network

Backpropagation starts with $\delta^{(L+1)} = \hat{y} - y$. Everything in §4 is about propagating that first δ downwards.

When one number per input is not enough

The three standard cases all assume the noise is the same everywhere. Two common situations where it is not.

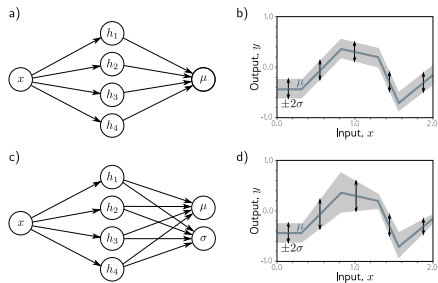
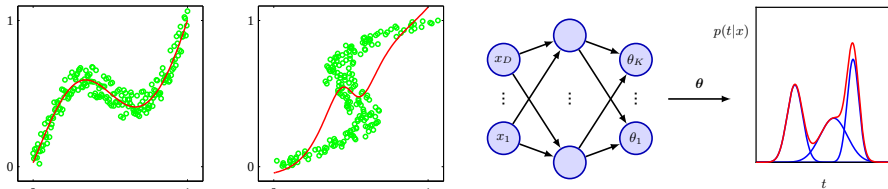


Figure from Prince (2023), Fig. 5.14: a network with a second output head predicting $\sigma(x)$, so the fitted noise width follows the data. The error function becomes $E_n = \frac{(y_n - \hat{\mu}_n)^2}{2\hat{\sigma}_n^2} + \ln \hat{\sigma}_n$, with $\hat{\sigma} = \exp(\cdot)$ to keep it positive.

When the target is genuinely multi-valued



Figures from Bishop & Bishop (2024), Figs. 6.17 and 6.18. Left: a forward problem, one answer per input. Middle: the same data read backwards — one input now has three valid answers, and a network trained on squared error predicts their average, a value that is never correct. Right: a **mixture density network** outputs the parameters of a whole distribution instead.

$$\cdot p(y | \mathbf{x}) = \sum_k \pi_k(\mathbf{x}) \mathcal{N}(y | \mu_k(\mathbf{x}), \sigma_k^2(\mathbf{x}))$$

Bishop & Bishop, §6.5

What the mixture learns

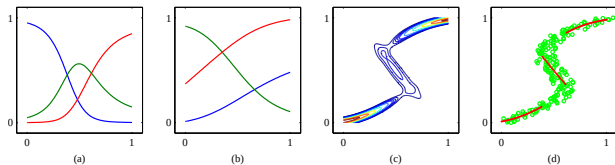


Figure from Bishop & Bishop (2024), Fig. 6.19: (a) the three mixing coefficients $\pi_k(x)$, (b) the three means $\mu_k(x)$, (c) the resulting conditional density $p(y | x)$, and (d) its modes — which track the three branches of the inverse problem instead of averaging them.

Backpropagation

The problem, stated

We have an error function and an architecture. Gradient descent needs one number per parameter, and there are W of them.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E(\mathbf{w}), \quad \nabla E = \left(\frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_W} \right)^\top$$

- A modern network has W in the millions or billions, and the gradient is needed every step
- Two obvious approaches — perturb each weight, or differentiate symbolically — both fail, for different reasons
- The way out is that E is a **composition**, and a composition can be differentiated once and reused

Why not just perturb each weight?

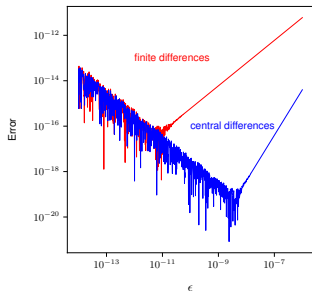


Figure from Bishop & Bishop (2024), Fig. 8.2: the error in a numerical derivative against the step size, for forward and central differences. Truncation falls with the step and rounding rises, so the best accuracy is poor — and each of the W derivatives needs its own forward pass.

- Still useful for **checking** an implementation on a small network — and only for that

The one idea

A weight $w_{ji}^{(l)}$ affects E **only through** the pre-activation $a_j^{(l)}$ it feeds. So the chain rule splits every derivative into two factors:

$$\frac{\partial E_n}{\partial w_{ji}^{(l)}} = \underbrace{\frac{\partial E_n}{\partial a_j^{(l)}}}_{\delta_j^{(l)}} \cdot \underbrace{\frac{\partial a_j^{(l)}}{\partial w_{ji}^{(l)}}}_{z_i^{(l-1)}} = \delta_j^{(l)} z_i^{(l-1)} \quad (\text{B\&B 8.3--8.5})$$

- $z_i^{(l-1)}$ is already known — computed on the way **up**
- So the problem reduces to the $\delta_j^{(l)}$: one per *unit* rather than one per *weight*, and they satisfy a recursion

FORWARD store every $z^{(l)}$

BACKWARD compute every $\delta^{(l)}$, then multiply

The backpropagation equations

Theorem 5 (Backpropagation)

For a feed-forward network with error E_n and differentiable activation h ,

$$\delta^{(l+1)} = \hat{\mathbf{y}}_n - \mathbf{y}_n, \quad \delta_j^{(l)} = h'(a_j^{(l)}) \sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)}, \quad \frac{\partial E_n}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} z_i^{(l-1)}.$$

- $a_j^{(l)}$ reaches E_n only through the units k it feeds: $\delta_j^{(l)} = \sum_k \frac{\partial E_n}{\partial a_k^{(l+1)}} \frac{\partial a_k^{(l+1)}}{\partial a_j^{(l)}}$
- The first factor is $\delta_k^{(l+1)}$ by definition
- For the second, $a_k^{(l+1)} = \sum_j w_{kj}^{(l+1)} h(a_j^{(l)})$, so $\partial a_k^{(l+1)} / \partial a_j^{(l)} = w_{kj}^{(l+1)} h'(a_j^{(l)})$
- Substituting and pulling h' out of the sum gives the recursion; the top-layer value is the canonical-link theorem of §3

□

The backpropagation equations

Theorem 5 (Backpropagation)

For a feed-forward network with error E_n and differentiable activation h ,

$$\delta^{(l+1)} = \hat{\mathbf{y}}_n - \mathbf{y}_n, \quad \delta_j^{(l)} = h'(a_j^{(l)}) \sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)}, \quad \frac{\partial E_n}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} z_i^{(l-1)}.$$

- $a_j^{(l)}$ reaches E_n only through the units k it feeds: $\delta_j^{(l)} = \sum_k \frac{\partial E_n}{\partial a_k^{(l+1)}} \frac{\partial a_k^{(l+1)}}{\partial a_j^{(l)}}$
- **The first factor is $\delta_k^{(l+1)}$ by definition**
- For the second, $a_k^{(l+1)} = \sum_j w_{kj}^{(l+1)} h(a_j^{(l)})$, so $\partial a_k^{(l+1)} / \partial a_j^{(l)} = w_{kj}^{(l+1)} h'(a_j^{(l)})$
- Substituting and pulling h' out of the sum gives the recursion; the top-layer value is the canonical-link theorem of §3

□

The backpropagation equations

Theorem 5 (Backpropagation)

For a feed-forward network with error E_n and differentiable activation h ,

$$\delta^{(L+1)} = \hat{y}_n - y_n, \quad \delta_j^{(l)} = h'(a_j^{(l)}) \sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)}, \quad \frac{\partial E_n}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} z_i^{(l-1)}.$$

- $a_j^{(l)}$ reaches E_n only through the units k it feeds: $\delta_j^{(l)} = \sum_k \frac{\partial E_n}{\partial a_k^{(l+1)}} \frac{\partial a_k^{(l+1)}}{\partial a_j^{(l)}}$
- The first factor is $\delta_k^{(l+1)}$ by definition
- **For the second**, $a_k^{(l+1)} = \sum_j w_{kj}^{(l+1)} h(a_j^{(l)})$, so $\partial a_k^{(l+1)} / \partial a_j^{(l)} = w_{kj}^{(l+1)} h'(a_j^{(l)})$
- Substituting and pulling h' out of the sum gives the recursion; the top-layer value is the canonical-link theorem of §3

□

The backpropagation equations

Theorem 5 (Backpropagation)

For a feed-forward network with error E_n and differentiable activation h ,

$$\delta^{(l+1)} = \hat{y}_n - y_n, \quad \delta_j^{(l)} = h'(a_j^{(l)}) \sum_k w_{kj}^{(l+1)} \delta_k^{(l+1)}, \quad \frac{\partial E_n}{\partial w_{ji}^{(l)}} = \delta_j^{(l)} z_i^{(l-1)}.$$

- $a_j^{(l)}$ reaches E_n only through the units k it feeds: $\delta_j^{(l)} = \sum_k \frac{\partial E_n}{\partial a_k^{(l+1)}} \frac{\partial a_k^{(l+1)}}{\partial a_j^{(l)}}$
- The first factor is $\delta_k^{(l+1)}$ by definition
- For the second, $a_k^{(l+1)} = \sum_j w_{kj}^{(l+1)} h(a_j^{(l)})$, so $\partial a_k^{(l+1)} / \partial a_j^{(l)} = w_{kj}^{(l+1)} h'(a_j^{(l)})$
- **Substituting and pulling h' out of the sum gives the recursion; the top-layer value is the canonical-link theorem of §3**

□

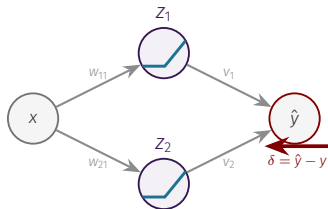
The algorithm

Algorithm 6 (Backpropagation for one example)

1. *Forward.* Set $\mathbf{z}^{(0)} = \mathbf{x}_n$ and for $l = 1, \dots, L$ compute $\mathbf{a}^{(l)} = \mathbf{W}^{(l)}\mathbf{z}^{(l-1)} + \mathbf{b}^{(l)}$ and $\mathbf{z}^{(l)} = h(\mathbf{a}^{(l)})$. *Store every $\mathbf{a}^{(l)}$ and $\mathbf{z}^{(l)}$.*
2. *Output error.* $\delta^{(L+1)} = \hat{\mathbf{y}}_n - \mathbf{y}_n$.
3. *Backward.* For $l = L, \dots, 1$, $\delta^{(l)} = h'(\mathbf{a}^{(l)}) \odot (\mathbf{W}^{(l+1)\top} \delta^{(l+1)})$.
4. *Derivatives.* $\partial E_n / \partial \mathbf{W}^{(l)} = \delta^{(l)} \mathbf{z}^{(l-1)\top}$ and $\partial E_n / \partial \mathbf{b}^{(l)} = \delta^{(l)}$.

Sum over n for the full-batch gradient.

A worked pass, by hand



Forward

$$a_j = w_{j1}x, \quad z_j = h(a_j)$$

$$a = v_1 z_1 + v_2 z_2, \quad \hat{y} = a$$

Backward

$$\delta = \hat{y} - y$$

$$\delta_j = h'(a_j) v_j \delta$$

$$\frac{\partial E}{\partial v_j} = \delta z_j, \quad \frac{\partial E}{\partial w_{j1}} = \delta_j x$$

Four derivatives, one forward pass and one backward pass — no derivative is ever computed twice.

What it costs

Proposition 7 (Backpropagation is $O(W)$)

For a network with W weights, one forward pass costs $O(W)$ multiply-adds, and the backward pass costs another $O(W)$. The complete gradient therefore costs $O(W)$ — a small constant times the cost of one forward evaluation.

Proof. Each weight is used exactly once in the forward pass (in the sum forming its target's pre-activation) and exactly once in the backward pass (in the sum forming its source's δ). Every unit contributes one evaluation of h and one of h' , and there are fewer units than weights. $\square$

- Compare $O(W^2)$ for finite differences: at $W = 10^6$ that is a factor of a million
- This single fact is why gradient-based training of large networks is possible at all

The recursion has a price: vanishing and exploding

The backward pass multiplies by h' and by W^T once per layer. Repeated multiplication is unstable in both directions.

Proposition 8 (Gradient scaling across layers)

$\delta^{(l)} = (\prod_{m=l+1}^L D^{(m)} W^{(m)T}) \delta^{(L+1)}$ with $D^{(m)} = \text{diag } h'(a^{(m)})$, so

$\|\delta^{(l)}\| \leq \prod_m \|W^{(m)}\| \sup |h'| \cdot \|\delta^{(L+1)}\|$: the bound is a *product of $L - l$ factors* and is therefore exponential in depth unless those factors sit near 1.

- Factor below 1: the gradient *vanishes* and early layers stop learning
- Factor above 1: it *explodes* and the step overshoots
- Fixes are Classes 12–15: careful initialisation, normalisation, residual connections, gradient clipping

Measured, over twenty layers

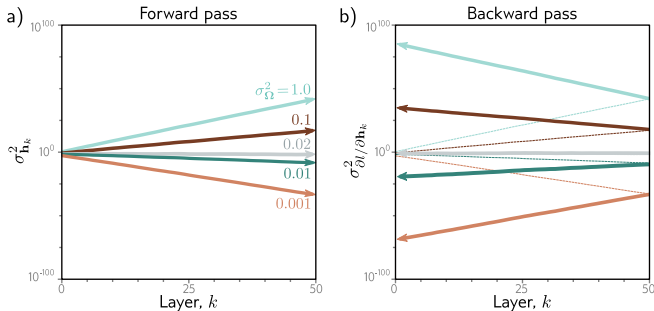


Figure from Prince (2023), Fig. 7.7: (a) the forward pass and (b) the backward pass through a deep network, plotted layer by layer. Both grow or shrink geometrically with depth, and only a narrow band of initialisation scales keeps them near constant.

Automatic Differentiation

Three ways to differentiate a program

Backpropagation was derived for one architecture. Real networks branch, share weights and change shape — so we want the general version.

Method	How	Why it is not enough
Numerical	perturb and difference	$O(W^2)$, and only 8 digits at best
Symbolic	manipulate the formula	expression swell; needs a closed form
Automatic	apply the chain rule to the computational graph	—

Automatic differentiation is **exact** to machine precision, works on any program with branches and loops, and costs a constant times the function itself.

The computational graph

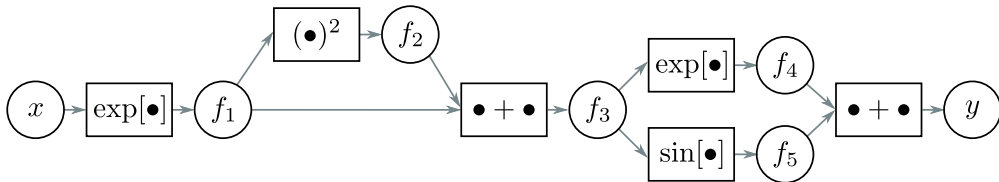


Figure from Prince (2023), Fig. 7.8: a program that computes a scalar output, drawn as a directed acyclic graph of elementary operations. Each node knows its own local derivative, which is all automatic differentiation needs. *Forward mode* pushes derivatives along the arrows; *reverse mode* pulls them back.

Forward mode

Carry a **tangent** $\dot{v} = \partial v / \partial x_d$ alongside every value, for one chosen input x_d :

$$v = f(u) \implies \dot{v} = f'(u) \dot{u}, \quad v = u_1 u_2 \implies \dot{v} = \dot{u}_1 u_2 + u_1 \dot{u}_2$$

- Seed $\dot{x}_d = 1$ and all other inputs' tangents to 0, then sweep **forwards** once
- One sweep gives $\partial v / \partial x_d$ for **every** intermediate v — a whole column of the Jacobian
- Implemented neatly with **dual numbers** $u + \dot{u} \epsilon$ where $\epsilon^2 = 0$
- Cost: **D sweeps** for D inputs

FORWARD one input at a time

BACKWARD one output at a time — and we have only one output

Reverse mode

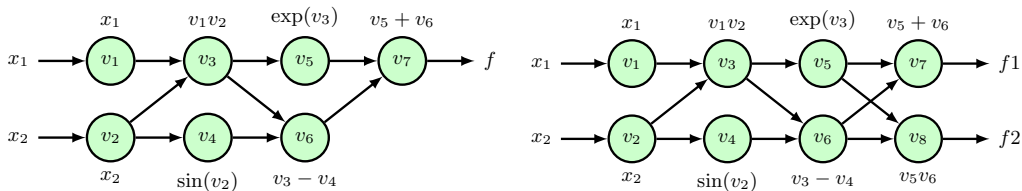
Carry an **adjoint** $\bar{v} = \partial E / \partial v$ instead, and sweep the other way:

$$\bar{u} = \sum_{v \in \text{children}(u)} \bar{v} \frac{\partial v}{\partial u}$$

- Seed $\bar{E} = 1$ and sweep **backwards** once
- One sweep gives $\partial E / \partial v$ for **every** node — a whole *row* of the Jacobian, which for a scalar E is the entire gradient
- Needs the forward values, so they must be **stored** on the way up
- Cost: K **sweeps** for K outputs — and $K = 1$

Backpropagation is reverse-mode AD on a layered graph: the $\delta_j^{(l)}$ are the adjoints $\bar{a}_j^{(l)}$.

Which mode, and why



Figures from Bishop & Bishop (2024), Figs. 8.4 and 8.5: the same evaluation trace differentiated in forward mode (left) and reverse mode (right). Training a network means $D = W$ inputs and $K = 1$ output, which puts us as far into reverse-mode territory as it is possible to be.

The cheap gradient principle

Theorem 9 (Baur & Strassen, 1983)

Let $f : \mathbb{R}^D \rightarrow \mathbb{R}$ be computed by a straight-line program of C arithmetic operations. Then the full gradient ∇f can be computed in $O(C)$ operations — in fact at most about $4C$, independent of D .

- A gradient with a million components costs no more than about four function evaluations
- The $O(W)$ cost of backpropagation is a **special case** of this, not a property of neural networks
- The price is **memory**: reverse mode must retain the forward values, so storage grows with the graph

What it costs in memory, and how to pay less

- The backward pass needs every stored $\mathbf{a}^{(l)}$ and $\mathbf{z}^{(l)}$: memory grows **linearly with depth** and with the batch size
- **Gradient checkpointing**: keep only every $\sqrt{L}$ -th layer's activations and recompute the rest during the backward pass — memory $O(\sqrt{L})$ for about one extra forward pass
- **Reversible layers** go further: reconstruct $\mathbf{z}^{(l-1)}$ from $\mathbf{z}^{(l)}$ and store nothing
- This trade — recompute versus retain — is why very deep models are trainable on finite hardware

FORWARD cheap in time, expensive in memory

BACKWARD checkpointing trades one back for the other

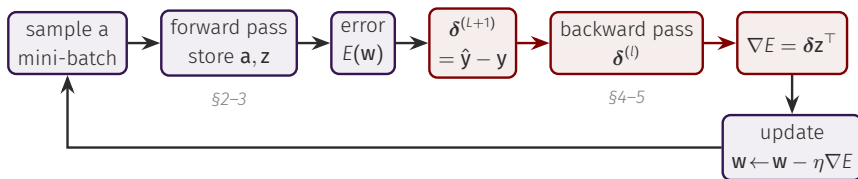
How the frameworks do it

- **Tape-based (PyTorch)**. Every operation on a tracked tensor appends a node recording its inputs and its local derivative. Calling **backward()** walks the tape in reverse. The graph is built afresh each iteration, so branches and loops are free.
- **Trace-and-transform (JAX)**. The function is traced once into a graph, then a program transformation (**grad**) produces a new program computing the gradient, which is then compiled.
- Both are reverse-mode AD. Neither contains anything specific to neural networks — a network is just one program among many.

You will never implement backpropagation. You will regularly **debug** it — which needs the equations in §4.

Putting It Together

One training step, end to end



- Everything in maroon is one call to **backward()**
- The update rule itself — how to choose η , and what to do instead of plain descent — is **Class 13**

Takeaways

1. An activation is chosen for its **derivative**: the logistic caps h' at $\frac{1}{4}$; ReLU does not saturate but its units can **die permanently**
2. Pairing an output activation with the matching error function is the **canonical link**, and it makes $\partial E / \partial a = \hat{y} - y$ — one theorem behind the three cases of Class 6
3. Error functions extend past those three: **heteroscedastic** outputs predict $\sigma(x)$, **mixture density** outputs keep multi-valued answers that squared error would average away
4. **Backpropagation** splits every derivative as $\delta_j^{(l)} z_i^{(l-1)}$ and gets all W of them in $O(W)$ — against $O(W^2)$ for finite differences
5. It is **reverse-mode automatic differentiation** on the computational graph, and by Baur–Strassen the gradient costs a small constant times the function, at the price of memory

Class 13: we can compute ∇E cheaply and exactly. Next, what to **do** with it — gradient descent and its modern variants.

References

- C. M. Bishop & H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. §6.2–6.3 (activation functions), §6.4 (error functions and the canonical link), §6.5 (mixture density networks), Ch. 8 (backpropagation, §8.1; automatic differentiation, §8.2).
- S. J. D. Prince. *Understanding Deep Learning*. MIT Press, 2023. Ch. 5 (loss functions, including heteroscedastic and mixture models), Ch. 7 (gradients, backpropagation, initialisation and the exploding-gradient problem).
- W. Baur & V. Strassen, *Theoretical Computer Science* 22:317 (1983) — the complexity of partial derivatives: the “cheap gradient” result.
- D. Rumelhart, G. Hinton & R. Williams, *Nature* 323:533 (1986) — learning representations by back-propagating errors.
- A. Baydin, B. Pearlmutter, A. Radul & J. Siskind, *JMLR* 18:153 (2018) — automatic differentiation in machine learning: a survey.
- Figures 6.12, 6.17–6.19, 8.1–8.5 reproduced from Bishop & Bishop (2024); Figures 5.14 and 7.7–7.8 from Prince (2023), <https://udlbook.github.io/udlbook/>, CC BY-NC-ND. The three remaining plots have no textbook counterpart and are generated by `../B_Recreated_Figures/src/build_all.py`.

Questions?